CENTIPEDE GAME WEB DEVELOPMENT PROJECT

Technical Proposal & Implementation Plan

Team Members: Muhammad Musa Khan (23i-0526) Muhammad Ali (23i-0008)

1. EXECUTIVE SUMMARY

This proposal outlines the comprehensive development plan for creating a web-based implementation of the classic arcade game Centipede. The project will utilize modern web technologies including HTML5, CSS3, and JavaScript to deliver an engaging, responsive gaming experience that captures the essence of the original while incorporating contemporary web standards and best practices.

Project Objectives:

- Develop a fully functional, browser-based Centipede game
- Implement responsive design for cross-device compatibility
- Ensure smooth gameplay with optimized performance
- Create an intuitive user interface with modern aesthetics
- Deploy a scalable, maintainable codebase

2. PROJECT OVERVIEW

2.1 Game Description

Centipede is a fixed-shooter arcade game where the player controls a shooter at the bottom of the screen, defending against a descending centipede and other enemies. The game features:

- A segmented centipede that moves down the playing field
- Mushrooms that serve as obstacles and scoring elements
- Additional enemies including spiders, fleas, and scorpions
- Progressive difficulty with increasing speed and complexity
- Score tracking and level progression system

2.2 Technical Scope

The project will be developed using pure web technologies without external frameworks, ensuring lightweight delivery and broad compatibility:

- HTML5 Canvas for game rendering
- Vanilla JavaScript for game logic and mechanics
- CSS3 for styling and responsive layout
- LocalStorage API for score persistence

- RequestAnimationFrame for smooth animations

3. TECHNICAL ARCHITECTURE

3.1 System Architecture

The application will follow a modular, object-oriented architecture with clear separation of concerns:

- Game Engine Core: Main game loop, state management, and rendering pipeline
- Entity System: Base classes for all game objects (player, enemies, projectiles)
- Input Handler: Keyboard and touch input processing
- Collision Detection: Efficient spatial partitioning for collision checks
- Rendering Layer: Canvas-based graphics rendering with sprite management
- Audio Manager: Sound effects and background music control
- Score System: Points calculation, high score tracking, and persistence

3.2 File Structure

Organized project structure for maintainability:

```
centipede-game/
├── index.html          # Main game page
├── css/
│   ├── main.css        # Primary styles
│   ├── game.css        # Game-specific styles
│   └── responsive.css  # Media queries
├── js/
│   ├── main.js         # Application entry point
│   ├── game.js         # Core game engine
│   ├── player.js       # Player entity
│   ├── centipede.js    # Centipede entity
│   ├── enemies.js      # Other enemies (spider, flea, scorpion)
│   ├── projectile.js   # Bullet mechanics
│   ├── mushroom.js     # Obstacle management
│   ├── collision.js    # Collision detection
│   ├── input.js        # Input handling
│   ├── renderer.js     # Canvas rendering
│   ├── audio.js        # Sound management
│   └── utils.js        # Helper functions
├── assets/
│   ├── sprites/        # Game graphics
│   ├── sounds/         # Audio files
│   └── fonts/          # Custom fonts
└── README.md           # Documentation
```

4. IMPLEMENTATION DETAILS

4.1 HTML5 Canvas Implementation

The game will be rendered on an HTML5 Canvas element with the following specifications:

- Canvas Resolution: 800x600 pixels (maintaining classic aspect ratio)
- Scaling: Dynamic scaling to fit viewport while maintaining aspect ratio
- Double Buffering: Off-screen canvas for flicker-free rendering
- Coordinate System: Top-left origin with normalized game coordinates
- Frame Rate: Target 60 FPS using requestAnimationFrame

4.2 Game Loop Architecture

The core game loop will follow the standard update-render pattern:

1. Input Processing: Capture and queue input events
2. Update Phase:
   - Update player position based on input
   - Update all enemy positions and behaviors
   - Update projectiles
   - Check collisions
   - Update game state
3. Render Phase:
   - Clear canvas
   - Render background
   - Render mushrooms
   - Render enemies
   - Render player
   - Render projectiles
   - Render UI elements
4. Timing: Calculate delta time for frame-independent movement

4.3 Entity System Design

All game entities will inherit from a base GameObject class:

Base GameObject Class:
- Properties: x, y, width, height, velocity, active
- Methods: update(), render(), checkCollision(), destroy()

Player Class:
- Additional Properties: lives, score, fireRate, invulnerable
- Methods: move(), shoot(), takeDamage(), respawn()

Centipede Class:
- Properties: segments[], direction, speed, poisoned

- Methods: move(), split(), destroy(), changeDirection()

Enemy Classes (Spider, Flea, Scorpion):
- Unique movement patterns and behaviors
- Collision effects and point values

Mushroom Class:
- Properties: health, poisoned, points
- Methods: takeDamage(), poison(), regenerate()

4.4 Collision Detection System

Efficient collision detection using spatial partitioning:

- Grid-Based Partitioning: Divide playing field into cells for broad-phase detection
- AABB Collision: Axis-aligned bounding box for precise collision checks
- Collision Pairs: Player-Enemy, Projectile-Enemy, Projectile-Mushroom
- Optimization: Only check active entities within relevant grid cells
- Response: Immediate feedback with visual and audio cues

4.5 Input Handling

Multi-platform input support for accessibility:

- Keyboard Controls: Arrow keys for movement, Space for shooting
- Mouse Control: Click to move, click to shoot (optional)
- Touch Controls: Virtual joystick and fire button for mobile devices
- Input Buffering: Queue inputs to prevent loss during frame drops
- Configurable Keys: Allow users to customize control scheme

5. GAME MECHANICS

5.1 Centipede Movement Algorithm

The centipede follows a distinctive zigzag pattern:

1. Initial State: Centipede enters from top-left, moving right
2. Horizontal Movement: Move in current direction until obstacle or edge
3. Descent Trigger: Upon hitting mushroom or screen edge
4. Descent Behavior: Drop down one row and reverse direction
5. Segmentation: When shot, split into multiple independent centipedes
6. Head Creation: Each split creates new head segments
7. Speed Increase: Shorter centipedes move faster
8. Bottom Behavior: Upon reaching bottom area, move erratically upward

5.2 Enemy Behaviors

Spider:

- Random diagonal movement across lower portion of screen
- Can destroy mushrooms in its path
- Variable point value based on proximity to player when killed
- Unpredictable pattern creates dynamic challenge

Flea:

- Falls vertically from top when few mushrooms remain in player area
- Drops mushrooms as it descends
- Requires multiple hits to destroy
- Helps replenish mushroom field

Scorpion:

- Travels horizontally across middle of screen
- Poisons mushrooms it touches
- Poisoned mushrooms cause centipede to plunge downward
- High point value encourages strategic shooting

5.3 Scoring System

Point values for game elements:

Game Element	Point Value
Centipede Body Segment	10 points
Centipede Head Segment	100 points
Spider (far)	300 points
Spider (medium)	600 points
Spider (close)	900 points
Flea	200 points
Scorpion	1000 points

5.4 Level Progression

Difficulty scaling across levels:

- Level 1-3: Standard speed, single centipede
- Level 4-6: Increased speed, additional spider appearances
- Level 7-9: Multiple centipedes, faster enemy spawn rates
- Level 10+: Maximum speed, all enemies active, dense mushroom fields
- Bonus Life: Awarded every 10,000 points
- Wave Clear Bonus: 1000 points for clearing all centipede segments

6. CSS IMPLEMENTATION

6.1 Layout Design

Responsive layout with flexbox/grid for modern browsers:

- Centered game container with max-width constraints
- Flexible canvas sizing maintaining 4:3 aspect ratio
- Score display and UI elements positioned absolutely
- Start screen, pause overlay, and game over screens
- Mobile-first design approach

6.2 Visual Theme

Retro-modern aesthetic honoring the classic:

- Color Scheme: Dark background with vibrant neon accents
- Typography: Monospace fonts for authentic arcade feel
- Animations: CSS transitions for UI state changes
- Pixel-Perfect: Crisp rendering with image-rendering: pixelated
- Accessibility: High contrast ratios, keyboard navigation support

6.3 Responsive Breakpoints

Device Type	Breakpoint	Adaptations
Mobile Portrait	< 480px	Scaled canvas, touch controls
Mobile Landscape	480px - 768px	Full-width canvas, optimized UI
Tablet	768px - 1024px	Larger canvas, hybrid controls
Desktop	> 1024px	Maximum canvas size, keyboard focus

7. JAVASCRIPT ARCHITECTURE

7.1 Code Organization

Modular JavaScript with ES6+ features:

- ES6 Classes: Object-oriented structure for game entities
- Modules: ES6 import/export for code organization
- Arrow Functions: Concise syntax for callbacks
- Destructuring: Clean parameter handling
- Template Literals: Dynamic string generation
- Constants: Centralized configuration object
- Strict Mode: Enforce best practices

7.2 Performance Optimization

Techniques to ensure smooth 60 FPS gameplay:

- Object Pooling: Reuse projectile and particle objects
- Dirty Rectangle: Only redraw changed canvas regions (optional)
- RequestAnimationFrame: Browser-optimized rendering loop
- Delta Time: Frame-rate independent movement
- Spatial Partitioning: Reduce collision check complexity from $O(n^2)$ to $O(n)$
- Lazy Loading: Load assets on demand
- Debouncing: Prevent input spam
- Web Workers: Offload heavy calculations (if needed)

7.3 State Management

Game state machine with clear transitions:

States:
- LOADING: Asset loading and initialization
- MENU: Start screen with options
- PLAYING: Active gameplay
- PAUSED: Game paused, overlay visible
- GAME_OVER: End screen with score and restart option
- LEVEL_TRANSITION: Brief animation between levels

Transitions:
- LOADING → MENU (assets loaded)
- MENU → PLAYING (start button clicked)
- PLAYING ↔ PAUSED (pause key pressed)
- PLAYING → GAME_OVER (lives == 0)
- PLAYING → LEVEL_TRANSITION (centipede cleared)
- LEVEL_TRANSITION → PLAYING (animation complete)
- GAME_OVER → MENU (restart clicked)

7.4 Data Persistence

LocalStorage implementation for game data:

- High Scores: Store top 10 scores with timestamps
- Settings: Volume levels, control preferences, difficulty
- Progress: Last completed level (optional continue feature)
- Statistics: Total games played, total score, best streak
- Validation: Check for corrupted data and provide defaults

8. AUDIO IMPLEMENTATION

8.1 Sound Effects

HTML5 Audio API for game sounds:

- Player Shoot: Short laser sound for each bullet fired
- Enemy Hit: Distinct sound for each enemy type destroyed
- Mushroom Hit: Soft impact sound
- Player Death: Dramatic explosion effect
- Level Complete: Victory jingle
- Centipede Movement: Subtle crawling sound (looped)
- Spider Jump: Quick whoosh sound

8.2 Audio Manager

Centralized audio control system:

- Sound Pool: Pre-load and pool audio instances for rapid playback
- Volume Control: Master volume and individual channel controls
- Mute Toggle: Instant mute/unmute functionality
- Audio Sprites: Single file with multiple sounds for efficiency
- Fallback: Graceful degradation if audio unavailable
- Mobile Compatibility: Handle iOS/Android autoplay restrictions

9. TESTING & QUALITY ASSURANCE

9.1 Testing Methodology

Testing Type	Coverage
Unit Testing	Individual functions, collision detection, scoring logic
Integration Testing	Game loop, state transitions, entity interactions
Cross-Browser Testing	Chrome, Firefox, Safari, Edge (latest 2 versions)
Device Testing	Desktop, tablet, mobile (iOS/Android)

9.2 Performance Benchmarks

Target performance metrics:

- Frame Rate: Consistent 60 FPS on modern hardware
- Load Time: < 2 seconds on 3G connection
- Memory Usage: < 100MB RAM during gameplay
- CPU Usage: < 25% on mid-range devices
- Input Latency: < 16ms response time
- Battery Impact: Minimal drain on mobile devices

9.3 Quality Checklist

Pre-deployment verification:

- All game mechanics functioning correctly
- No collision detection bugs or glitches
- Smooth animations without stuttering
- Audio playing correctly on all tested devices
- Score calculation accurate
- High score persistence working
- Responsive design functional across breakpoints
- No console errors or warnings
- Accessible keyboard navigation
- Touch controls responsive on mobile
- Cross-browser compatibility verified

10. DEPLOYMENT STRATEGY

10.1 Hosting Requirements

Minimal infrastructure needs for static site:

- Static File Hosting: GitHub Pages, Netlify, or Vercel
- CDN: Optional CDN for faster global asset delivery
- HTTPS: SSL certificate for secure connections
- Caching: Aggressive caching for assets with versioning
- Compression: Gzip/Brotli compression for text files

10.2 Build Process

Optimization pipeline for production:

- Minification: Minify JavaScript and CSS files
- Image Optimization: Compress sprites and assets
- Bundling: Combine files to reduce HTTP requests
- Versioning: Add hash to filenames for cache busting
- Source Maps: Generate maps for debugging
- Testing: Run automated tests before deployment

10.3 Monitoring & Analytics

Post-launch tracking:

- Google Analytics: Track user engagement and sessions
- Error Tracking: Monitor JavaScript errors with service like Sentry
- Performance Monitoring: Track real-user metrics (RUM)
- User Feedback: Implement feedback mechanism
- A/B Testing: Test different difficulty curves or UI layouts

11. PROJECT TIMELINE

Estimated development schedule (4-6 weeks):

Phase	Duration	Deliverables
Planning & Design	3-5 days	Architecture docs, wireframes, asset list
Core Engine Setup	4-6 days	Game loop, canvas, basic rendering
Entity Implementation	5-7 days	Player, centipede, enemies, projectiles
Game Mechanics	4-6 days	Collision, scoring, level progression
UI Development	3-4 days	Menus, HUD, overlays, responsive design
Audio Integration	2-3 days	Sound effects, audio manager
Testing & Debug	4-6 days	Cross-browser, device testing, bug fixes
Deployment	2-3 days	Build optimization, hosting setup, launch

12. TECHNICAL REQUIREMENTS

12.1 Development Tools

- Code Editor: VS Code or similar with JavaScript extensions
- Version Control: Git with GitHub repository
- Browser DevTools: Chrome/Firefox developer tools
- Image Editor: For sprite creation/editing
- Audio Editor: For sound effect creation/editing
- Local Server: For testing (Live Server extension or similar)

12.2 Browser Support

Browser	Minimum Version
Chrome	90+
Firefox	88+
Safari	14+
Edge	90+
Mobile Safari (iOS)	14+

12.3 System Requirements

Minimum User System:

- Processor: Dual-core 1.5 GHz or equivalent
- RAM: 2 GB

- Display: 1024x768 minimum resolution
- Internet: Broadband connection for initial load
- JavaScript: Must be enabled

Recommended User System:

- Processor: Quad-core 2.0 GHz or better
- RAM: 4 GB or more
- Display: 1920x1080 or higher
- Internet: High-speed broadband
- Hardware Acceleration: Enabled

13. RISK MANAGEMENT

Risk	Impact	Mitigation Strategy
Performance Issues	Medium	Early profiling, object pooling, optimization
Cross-Browser Bugs	Medium	Regular testing, feature detection, polyfills
Scope Creep	High	Clear requirements, phase-based development
Mobile Compatibility	Low	Responsive design, touch testing
Asset Creation Delays	Low	Use placeholder assets, parallel development

14. FUTURE ENHANCEMENTS

Potential post-launch features:

- Multiplayer Mode: Local co-op or online competitive play
- Leaderboards: Global high score rankings
- Power-ups: Temporary abilities (rapid fire, shield, slow-mo)
- Custom Levels: Level editor for user-created challenges
- Achievements: Unlock system for completing challenges
- Mobile App: PWA or native wrapper for app stores
- Additional Game Modes: Survival, time attack, boss rush
- Visual Themes: Unlockable skins and color schemes
- Controller Support: Gamepad API integration
- Accessibility: Colorblind modes, difficulty options

15. CONCLUSION

This comprehensive proposal outlines a complete development plan for creating a professional, browser-based Centipede game using HTML5, CSS3, and JavaScript. The modular architecture ensures maintainability, while the performance optimizations

guarantee smooth gameplay across devices. The project leverages modern web standards while honoring the classic arcade experience.

By following this implementation plan, the development team will deliver a polished, engaging game that captures the spirit of the original Centipede while providing a contemporary web gaming experience. The phased development approach allows for iterative testing and refinement, ensuring a high-quality final product.

Upon completion, the game will be deployable on any static hosting platform, accessible to players worldwide, and ready for future enhancements based on user feedback and emerging web technologies.

16. APPENDIX

16.1 Key Configuration Constants

```
const CONFIG = {
  CANVAS_WIDTH: 800,
  CANVAS_HEIGHT: 600,
  TARGET_FPS: 60,
  PLAYER_SPEED: 200,
  BULLET_SPEED: 400,
  CENTIPEDE_SPEED: 80,
  MUSHROOM_ROWS: 20,
  MUSHROOM_COLS: 30,
  GRID_SIZE: 20,
  PLAYER_FIRE_RATE: 250,
  STARTING_LIVES: 3,
  BONUS_LIFE_SCORE: 10000
};
```

16.2 Reference Resources

- MDN Web Docs: HTML5 Canvas API documentation
- JavaScript.info: Modern JavaScript tutorials
- Game Programming Patterns: By Robert Nystrom
- HTML5 Game Development: Online tutorials and courses
- Centipede Arcade Original: For authentic gameplay reference